



Chapter 4 存储过程与触发器

SQL注入攻击



School of software Yunnan University

□ SQL注入就是通过把SQL命令插入到Web表单递交或输入域名或页面请求的查询字符串，最终达到欺骗服务器执行恶意的SQL命令，比如先前的很多影视网站泄露会员密码大多就是通过WEB表单递交查询字符暴出的，这类表单特别容易受到SQL注入式攻击。

□ 构造巧妙的SQL语句，例如：

```
statement := "SELECT * FROM users WHERE name =  
'" + userName + "'"; "
```



SQL注入攻击



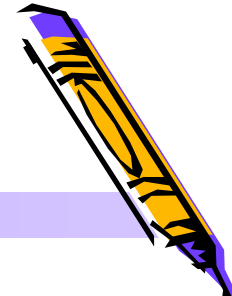
School of software Yunnan University

□ a' or 't'='t

□ SELECT * FROM users WHERE name = 'a' OR
't'='t';



SQL注入攻击



School of software Yunnan University

- ❑ statement := "SELECT * FROM users WHERE name = '' + userName + ''"; "
- ❑ a' or 't'='t; DROP TABLE users; SELECT * FROM data WHERE name LIKE '%
- ❑ SELECT * FROM users WHERE name = 'a'; DROP TABLE users; SELECT * FROM DATA WHERE name LIKE '%';



使用存储过程防止SQL注入



School of software Yunnan University

❑ create procedure GetUserInfoByUserName
(nvarchar(50) @UserName)

AS

select * from Users where UserName =
@UserName

■ ''; select * from master.dbo.sysdatabases; --"

■ ''; exec master.dbo.xp_cmdshell 'dir c:\ >
c:\filelist.txt'; --"

❑ 使用存储过程，这些字符串最后都会被当成字符串参数参与到最后的SQL查询中，数据库服务器把最后的SQL理解成：查询UserName字段的值为'; select * from master.dbo.sysdatabases; --'的数据，不可能找得到



不好的存储过程

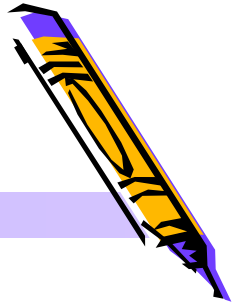


School of software Yunnan University

```
❑ CREATE PROCEDURE select_account  
  (@Account [char](16), @Pwd [char](32) )  
  AS  
  SELECT * FROM [Account] WHERE Account  
  t=@Account And Pwd=@Pwd
```



好的存储过程

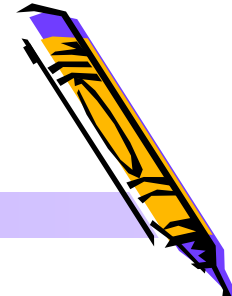


School of software Yunnan University

```
□ CREATE PROCEDURE select_account
  @Account char(16), @Pwd char(32)
  AS
  if exists(SELECT * FROM [Account] WHERE
    Account=@Account)
    begin
      if (select password from [Account] WHERE
        Account=@Account) = @Pwd
        begin select 1 end --成功登录
      else select 0 end --密码错误
    else select 2 ---无此用户
```



存储过程



School of software Yunnan University

□ 概述

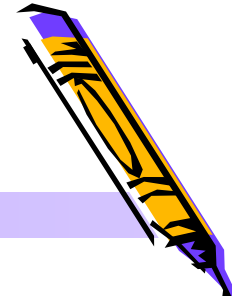
- **存储过程**就是作为可执行对象存放在数据库中的一个或多个**SQL**命令。存储过程可以交互式调用，可以在客户端应用程序代码中调用，在另一存储过程中调用或在触发器中调用。
- 存储过程可以传入参数和返回数值，从而增加可用性和灵活性。
- 存储过程还可以返回多个结果集和状态码

□ 存储过程的优点

- **模块化编程**—正如**C**、**C++**中常使用函数和子例程将代码分解成更小的块一样，使用存储过程可以得到同样的好处，只是存储过程存放在**DBMS**中，可以在任何客户端应用程序代码中调用。



存储过程



School of software Yunnan University

□ 存储过程的优点

- **基于函数限制访问表格**—可以访问执行存储过程而不必拥有直接操作基础表的权限
- **减少网络通信流**—存储过程可以包含多条**SQL**语句，但只要用一条语句来执行该存储过程，从而减少了客户端应用程序对服务器的调用次数和长度
- **快速执行**—存储过程在第一次执行时进行语法检查和编译，编译好的版本存储在高速缓存中，用于再次调用
- **保证一致性**—如果用户只通过存储过程修改数据，则可以消除偶然修改带来的问题
- **减少操作人员和编程人员的错误**—由于传递信息少，因此执行复杂任务更容易，不易出现**SQL**错误



存储过程



School of software Yunnan University

□ 存储过程的缺点（取决于环境）

- **编程语言功能差**—**SQL**语言不是很强大的或高度结构化的语言
- **与编程环境集成不够**—许多大型软件开发项目用工具进行版本处理、调试、复用等，这些工具可能不支持存储过程中的代码
- **移植性差**—**ANSI**标准和**DBMS**标准存在差异，许多**DBMS**厂家不支持新标准，导致存储过程移植性差



存储过程



School of software Yunnan University

□ 生成与执行存储过程

■ 格式:

```
create proc [owner.]procedure_name [; number]
[[([@parm_name datatype=default_value [output]
  [...][)])]
[with { recompile | encryption}]
As
SQL statements
[return {integer_status_value}]
```



存储过程



School of software Yunnan University

□ 生成与执行存储过程

- 例1：生成返回书名与作者名的存储过程

```
Create Proc dbo.title_authors AS
```

```
Select a.au_lname, a.au_fname, t.title
```

```
From titles t Inner Join
```

```
titleauthor ta On t.title_id=ta.title_id Right Outer Join
```

```
authors a On ta.au_id=a.au_id
```

```
Return
```

- 该存储过程中的查询涉及到三张表：**titles**、**titleauthor**和**authors**。**t**是表**titles**的别名、**a**是表**authors**的别名、**ta**是表**titleauthor**的别名。



存储过程



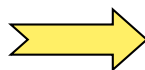
School of software Yunnan University

□ 生成与执行存储过程

- 执行存储过程：要执行存储过程，只要调用存储过程名即可。
- 如果过程不是批命令中的第一条语句，则要在过程名前面加上关键字**EXEC**。这是因为存储过程名不是关键字，当语句在执行过程中遇到存储过程名时，会将其作为上一条语句的一部分处理，因此导致执行出错。
- 例2：执行存储过程**title_authors**:

- title_authors
- Select * from titles
- title_authors
- Select * from titles
- **exec** title_authors

修改为



错，因为**title_authors**不是SQL定义的关键字，所以分析器将其解释成了**titles**表的别名，从而导致执行结果出错



存储过程



School of software Yunnan University

□ 生成与执行存储过程

```
select * from titles  
exec dbo.title_authors
```

```
CREATE PROC dbo.title_authors AS  
SELECT a.au_lname,  
       a.au_fname,  
       t.title  
FROM titles t  
      INNER JOIN  
      titleauthor ta  
ON t.title_id = ta.title_id  
      RIGHT OUTER JOIN authors a  
ON ta.au_id = a.au_id  
RETURN
```

结果		
title_id	title	
1	高等数学	
2	线性代数	
3	离散数学	
4	数据结构	
5	计算机组成原理	
6	计算机网络	
7	操作系统	

au_lname	au_fname	title
Gates	Bill	高等数学
Gates	Bill	计算机网络
Zuckerberg	Mark	线性代数
Zuckerberg	Mark	离散数学
Musk	Elon	数据结构
Musk	Elon	计算机...
Musk	Elon	操作系统



存储过程



School of software Yunnan University

□ 存储过程组

- **SQL Server**允许生成多个同名的存储过程，只要组号不同。默认组号为**1**。
- 要生成不同组号的存储过程，只要加上**分号（；）**和一个**整数**
- **例3**：生成查询国籍为'**CA**'的作者名与书名的存储过程

Create Proc dbo.title_authors; 2 AS

Select a.au_lname, a.au_fname, state, t.title

From titles t Inner Join

titleauthor ta On t.title_id=ta.title_id Right Outer Join

authors a On ta.au_id=a.au_id

Where a.state='CA'

Return



存储过程



School of software Yunnan University

□ 删除存储过程

- 格式:

Drop Proc *Procedure_name*

- 例如: 删除存储过程title_authors

Drop proc title_authors

□ 显示存储过程引用的对象

- 格式:

[exec] **sp_depends** {*procedure_name* | *table_name* }

- 显示存储过程引用的对象或引用数据表的数据库对象。

- 例如:

exec **sp_depends** title_authors



存储过程



School of software Yunnan University

□ 浏览与修改存储过程

- 存储过程源代码保存在系统表**syscomments**的**text**列中，可以直接查询该列获取信息。也可以使用系统存储过程**sp_helptext**获取存储过程的源代码
- 例如：exec **sp_helptext** title_authors
- 如果开发人员不让用户浏览源代码，则可以在生成存储过程时加上**With Encryption**选项，则源代码经过加密处理。
- 注意：**SQL Server**可以内部解密源代码，但该方法使得用户或任何最终用户工具都无法解密源代码



存储过程



School of software Yunnan University

□ 浏览与修改存储过程

- 修改存储过程: **Alter Proc**的语法和**Create Proc**语法相同, 但其优点是不必删除存储过程再重建, 因此, 也不必重新指定权限

- 例如:

```
Alter Proc dbo.title_authors
```

```
AS
```

```
Select a.au_lname 作者名, a.au_fname 作者姓, t.title 书名
```

```
From titles t Inner Join
```

```
titleauthor ta On t.title_id=ta.title_id Right Outer Join
```

```
authors a On ta.au_id=a.au_id
```

```
Return
```



存储过程



School of software Yunnan University

□ 浏览与修改存储过程

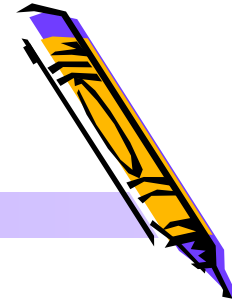
- 用**Alter Proc**修改存储过程的主要缺点是：数据库目录表中不记录过程改变的日期，因此**DBA**很难发现哪些过程是经过修改的。

□ 使用输入参数

- 要增加存储过程的灵活性或进行更复杂的处理，则可以将参数传入过程，参数可以在存储过程中使用本地变量的任何地方使用。
- 传递对象名作为参数



存储过程



School of software Yunnan University

□ 使用输入参数

■ 传递对象名作为参数

- Create proc find_data @table varchar(128)

as

exec ('select * from ' + @table)

return

■ 例2：执行存储过程find_data

exec find_data @table='authors'

在执行存储过程时为参数
@table赋值，即可实现对
不同数据表的查询



存储过程



School of software Yunnan University

□ 使用输入参数

■ 例3：设置参数的默认值

```
Create Proc dbo.title_authors2 @state char(2)='M%'
```

```
AS
```

```
Select a.au_lname 作者名, a.au_fname 作者姓, t.title 书名, a.state
```

```
From titles t Inner Join
```

```
titleauthor ta On t.title_id=ta.title_id Right Outer Join
```

```
authors a On ta.au_id=a.au_id
```

```
Where a.state like @state
```

```
Return
```



存储过程



School of software Yunnan University

□ 使用输出参数

- 如果调用批命令将变量作为参数传入存储过程，而参数在存储过程中被修改，则修改不会传给调用该存储过程的命令，除非生成和执行存储过程时对参数指定 **OUTPUT** 关键字
- 例1：根据给定书名，查询该书到目前为止的销量

- **Create proc** ytd_salespro
 @title varchar(80), @ytd_sale int **OUTPUT**
as
 select @ytd_sale=ytd_sales from titles
 where title=@title
return

- **含义：**从给定数据表titles中查找标题为@title的书籍到目前为止的销量，即在ytd_sales列上的取值，并将该值赋给输出参数@ytd_sale



存储过程



School of software Yunnan University

□ 使用输出参数

- 调用存储过程要声明一个存储返回值的变量，执行语句还要包括 **OUTPUT** 关键字，否则修改无法在调用中反映出来

- `Declare @sales_up_to_today int`

- `Exec ytd_salespro @title='Life Without Fear',`

- `@ytd_sale=@sales_up_to_today OUTPUT`

- `Print 'the output data is :`

- `' + convert(varchar(10),@sales_up_to_today)`

- **注意：**参数名（这里是@ytd_sale）在表达式左边列出，而本地变量（@sales_up_to_today）则设置为等于输出参数的值，在表达式右边列出。



存储过程



School of software Yunnan University

□ 使用输出参数

```
Create proc ytd_salespro
@title varchar(80),@ytd_sale int OUTPUT as
select @ytd_sale=ytd_sales from titles where title=@title
return
```

```
Declare @sales_up_to_today int
Exec ytd_salespro @title='高等数学',@ytd_sale=@sales_up_to_today OUTPUT
Print 'the output data is : '
      '+convert(varchar(10),@sales_up_to_today)
```

6
消息

the output data is :
999

完成时间: 2022-05-29T11:11:36.4008690+08:00



存储过程



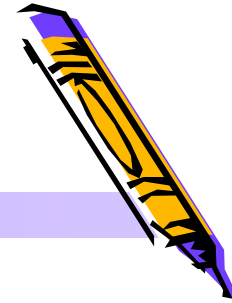
School of software Yunnan University

□ 使用输出参数

	title_id	title	ytd_sales
1	1	高等数学	999
2	2	线性代数	500
3	3	离散数学	600
4	4	数据结构	700
5	5	计算机组成原理	800
6	6	计算机网络	400
7	7	操作系统	666



存储过程



School of software Yunnan University

□ 返回过程状态

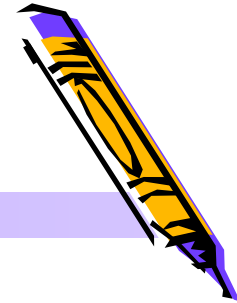
- 大多数编程语言能够向调用者返回过程状态，0通常表示成功，SQL Server也不例外。
- 例1：返回某本书到目前为止的销量，如果该书不存在，则返回状态值-101

```
Create Proc ytd_sales2 @title varchar(80)
as
if not exists(select * from titles where title=@title)
    return -101
select ytd_sales from titles
where title=@title
return
```

如果某本书的书名不存在，则返回
-101



存储过程



School of software Yunnan University

□ 返回过程状态

■ 例2：执行存储过程ytd_sales2

```
Declare @status int
```

```
exec @status=ytd_sales2 @title='Life without Fear'
```

```
if @status=-101
```

```
Print 'No title with that name found. '
```



存储过程

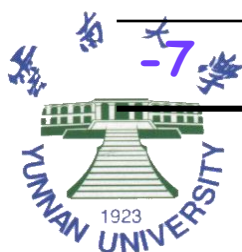


School of software Yunnan University

□ 返回过程状态

- **SQL Server** 的内部状态码。如果存储过程异常终止，则返回一个状态码，**SQL Server**保留了一1到一99的值。

状态码	含义	状态码	含义
0	执行成功	-8	存在非致命性内部错误
-1	缺对象	-9	达到系统极限
-2	存在数据类型错误	-10	存在致命内部不一致性
-3	选择该进程为死锁受害者	-11	存在致命内部不一致性
-4	存在权限错误	-12	表或索引混乱
-5	存在语法错误	-13	数据库混乱
-6	存在其他用户错误	-14	硬件故障
-7	存在资源错误，如空间不足		



存储过程



School of software Yunnan University

□ 实现数据插入

■ 插入一个新作者

Create Proc insertauthor

(@au_id varchar(11), @au_lname varchar(40), @au_fname
varchar(20), @phone char(12), @contract bit)

as

insert into authors(au_id, au_lname, au_fname, phone,
contract)

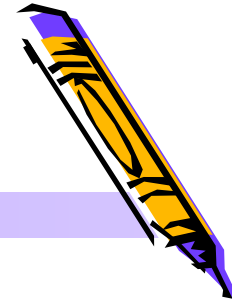
values(@au_id,@au_lname,@au_fname,@phone,@contract)

return

insertauthor au_id='000-00-0000', @au_lname='new',
@au_fname='one', @phone='555-555'555', @contract=0



存储过程



School of software Yunnan University

□ 在存储过程中使用临时表

- 中间结果要在工作表中存放，以便进行其他处理或更高级的处理时，通常要在存储过程中使用临时表
- **注意：**存储过程退出时，存储过程中生成的局部临时表自动删除，而全局临时表则仍然存在，直到显式删除，或生成临时表的用户与SQL Server断开



存储过程



School of software Yunnan University

□ 在存储过程中使用临时表

- 例如：生成存储过程，将**authors**表的查询信息保存到临时表**temp**中

```
Create proc temp_test
```

```
as
```

```
select * into ##temp
```

```
from authors
```

```
select * into #temp
```

```
from authors
```

```
return
```

```
go
```

```
exec temp_test
```

```
select * from ##temp
```



存储过程



School of software Yunnan University

□ 远程存储过程

- 可以使用四段命名方式执行另一服务器中的存储过程

`Exec server_name.db_name.owner_name.proc_name`

□ 系统存储过程

- 以 **sp_** 开头
- 存放在**master**数据库中
- 拥有者为**dbo**



触发器



School of software Yunnan University

□ 概述

- 触发器必须由事件才能触发，触发事件分为DML事件（**Insert**, **Update**, **Delete**）,DDL事件（**Create**, **Alter**, **Drop**）和数据库事件（**Startup**, **Shutdown**, **Logon**, **ServerError**）。

□ 触发器中不能使用的命令

Alter database	Create database
Disk init	Disk Resize
Drop database	Load database/load log
Reconfigure	Restore database/restore log



触发器



School of software Yunnan University

□ 触发器分为:

- **BEFORE**触发器（**SQL Server**不支持，**Oracle**等支持）在事件发生时触发。
- **AFTER**触发器是**SQL Server**生成的最初用于自动响应数据修改的机制。在**SQL Server2000**以前的版本中**AFTER**触发器是惟一的触发器，因此不用指明是**AFTER**，也可以用**for**代替，该种触发器的行为是在事件发生后执行。
- **SQL Server2000**中新增加了**INSTEAD OF**触发器，该触发器执行某项操作，而不是触发这个触发器的操作。



触发器 – After触发器



School of software Yunnan University

□ AFTER触发器

■ 格式:

Create TRIGGER *trigger_name*

ON table_name

AFTER {Insert | Update | Delete}

AS

SQL statement

■ AFTER触发器是默认的触发器类型，因此关键字**AFTER**可以选。



触发器 – After触发器



School of software Yunnan University

□ AFTER触发器

- 例1: 在pubs数据库中生成一个触发器, 打印一条消息, 表示 **update authors** 语句更新的数据行数。

```
Create TRIGGER tr_au_upd ON authors
```

```
AFTER UPDATE
```

```
as
```

```
Print 'Trigger Output: ' + convert(Varchar(5), @@rowcount) +  
' rows were updated. '
```

```
go
```

返回受上一语句影响的行数, 当没有行受影响时, 返回值为0



触发器 – After触发器



School of software Yunnan University

□ AFTER触发器

- 例1：通过执行**Update**语句查看tr_au_upd 触发器是否工作。

Update authors

Set au_fname=au_fname

Where state='UT'

go

Update titles

Set title=title

where ytd_sales=4095

go

实际上，我们并未
改变au_fname列的
内容，但还是触发
了触发器



触发器 – After触发器



School of software Yunnan University

□ AFTER触发器

- 例1：通过执行**Update**语句查看tr_au_upd 触发器是否工作。

```
CREATE TRIGGER tr_au_upd ON authors AFTER UPDATE
AS
PRINT 'Trigger Output: ' + convert(Varchar(5), @@rowcount) + 'rows were updated.'
GO

update authors set au_lname = 'Musk' where au_lname='Mark'
```

消息

Trigger Output: 1rows were updated.

(1 行受影响)



触发器 – After触发器



School of software Yunnan University

□ AFTER触发器

- 例1：通过执行**Update**语句查看tr_au_upd 触发器是否工作。

```
update authors set au_lname = 'Musk' where au_lname='Mark'
```

消息

Trigger Output: 0 rows were updated.

(0 行受影响)



触发器 – After触发器



School of software Yunnan University

□ inserted表与deleted表

- 在大多数触发器情况下，需要知道数据修改中发生了什么改变，可以在 **inserted** 与 **deleted** 表中找到这个信息

- 例如：

```
Select * INTO titles_copy From titles
```

```
go
```

```
Create Trigger tr_tc ON Titles_copy
```

```
For Insert, Update, Delete
```

```
AS
```

```
Print 'Inserted:'
```

```
Select title_id, type, price From inserted
```

```
Print 'Deleted:'
```

```
Select title_id, type, price From Deleted
```

```
return
```

修改后的行和要修改的行，分别复制到**Inserted**与**Deleted**表中



触发器 – After触发器



School of software Yunnan University

□ inserted表与deleted表

- 例如：触发触发器

Update titles_copy

Set price=\$15.05

Where type like '%cook%'

语句	inserted内容	deleted内容
Insert	增加的行	空
Update	新行	旧行
Delete	空	删除的行

Delete titles_copy

Where type like '%cook%'



触发器 – After触发器



School of software Yunnan University

□ inserted表与deleted表

```
Create Trigger tr_tc
```

```
ON titles
```

数据库中已存在名为“tr_tc”的对象。

```
For Insert , Update, Delete
```

```
AS
```

```
Print 'Inserted:'
```

```
select title_id, type, price
```

```
From inserted
```

```
Print 'Deleted:'
```

```
select title_id, type, price
```

```
From Deleted
```

```
return
```

```
update titles set price = 20  
where type = 1
```

5 %

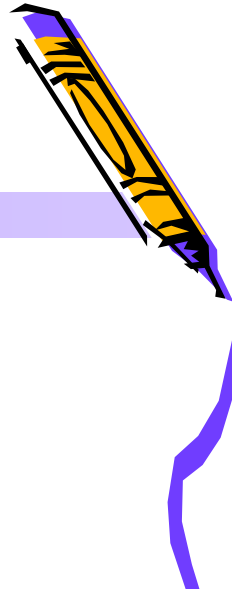
结果 消息

title_id	type	price
3	1	20
2	1	20
1	1	20

title_id	type	price
3	1	30
2	1	15
1	1	20



触发器 – After触发器



School of software Yunnan University

□ inserted表与deleted表

```
Create Trigger tr_tc
ON titles
For Insert , Update, Delete
AS
Print 'Inserted:'
select title_id, type, price
From inserted
Print 'Deleted:'
select title_id, type, price
From Deleted
return
```

数据库中已存在名为“tr_tc”的对象。

```
delete from titles where type = 3
```

146 %

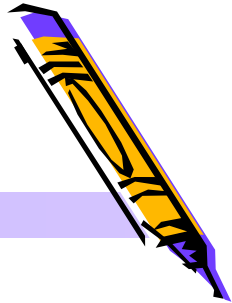
结果 消息

	title_id	type	price
--	----------	------	-------

	title_id	type	price
1	8	3	99



触发器 – After触发器



School of software Yunnan University

□ 检查列更新

- Insert或Update触发器内可以使用update()函数
- 使用update()函数检查Insert与Update操作对数据列的影响

```
Create Trigger tr_ins_upd ON titles_copy
```

```
For Insert, Update
```

```
AS
```

```
IF UPDATE(title_id)
```

```
begin
```

```
RAISERROR('You can not change the title_id. ',15,1)
```

```
end
```

```
return
```



触发器 – After触发器



School of software Yunnan University

□ 检查列更新

- **Insert**或**Update**触发器内可以使用**update()**函数
- 使用**update()**函数检查**Insert**与**Update**操作对数据列的影响

```
update titles_copy  
set title_id='bbb'  
where title_id='BU1111'
```

通常这样编写验证数据完整性的触发器。如果违反完整性约束，则向客户应用程序返回一条错误消息



触发器 - After

如果同一列中定义了限制和触发器，则违反限制会终止语句，不再执行触发器

School of software Yunnan University

□ 执行

■ 执行**AFTER**触发器之前发生的事件

- 限制处理—包括检查限制、唯一限制和主键限制
- 声明式引用—这些操作是外部键限制定义的，保证表间的正确关系
- 触发操作—即触发触发器的数据修改操作。这项操作发生在触发器执行之前，但结果要等到完成触发器操作之后才提交到数据库。

■ 触发顺序

- 可以有多个对应于**Insert**、**Update**、**Delete**的触发器，**SQL Server**可以指定第一个和最后一个触发器，但中间的触发器顺序则无法确定



触发器 – After触发器



School of software Yunnan University

□ 执行

■ 触发顺序

- **Sp_settriggerorder**过程是设置触发器顺序的工具
- 格式:

Sp_settriggerorder Trigger_name, 顺序, '操作'

其中: 顺序为First、Last或None

操作为Insert、Update或删除

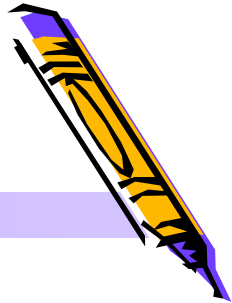
- 例如:

Sp_settriggerorder tr_au_upd2, **First**, 'Update'

- 建议尽量不要对同一个表的同一事件定义多个触发器, 可以把相关的所有操作定义到一个触发器中



触发器 – After触发器



School of software Yunnan University

□ 特殊考虑

- **AFTER**触发器可以用于具有级联参照完整性限制的表格中
- **WRITETEXT**与**TRUNCATE Table**不触发触发器
- 触发器是对象，因此要在数据库中有唯一的名称。

□ **AFTER**触发器的限制

- **AFTER**触发器只能用于表，不能用于视图
- 一个**AFTER**触发器不能用在多个表上
- **text**、**ntext**与**image**列不能引用**AFTER**触发器



触发器 – Instead Of 触发器



School of software Yunnan University

□ Instead of 触发器

- SQL Server 2000引入新的**Instead of**触发器
- **Instead of**的含义：该触发器执行某项操作，而不是触发这个触发器的操作。
- 语法格式：

Create Trigger *trigger_name*

ON *table_name*

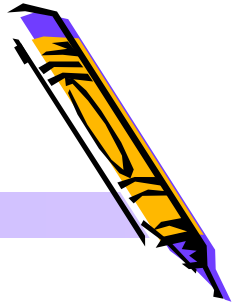
Instead of {insert | update | delete}

AS

SQL statements



触发器 – Instead Of 触发器



School of software Yunnan University

□ Instead of 触发器

■ 例:

```
create trigger tr_au_upd2 ON customers  
instead of Update
```

```
AS
```

```
print 'Trigger Output: '+Convert(varchar(5),  
@@rowcount)+'rows were updated.'
```

```
go
```



触发器 – Instead Of 触发器



School of software Yunnan University

□ Instead of 触发器

Update authors

Set au_fname=au_fname

Where state='UT'

- 更新操作并未执行，**Instead of**触发器的作用是打印一条消息
- **Instead of**触发器的执行细节：
 - 触发操作—触发**Instead of**触发器而不是触发操作
 - 限制处理—对限制的检查发生在**Instead of**触发器操作之后



触发器 – Instead Of 触发器



School of software Yunnan University

□ Instead of 触发器

```
create trigger tr_au_upd2
ON authors
instead of Update
AS
print 'Trigger Output: ' + Convert(varchar(5), @@rowcount) + 'rows were updated.'
go
```

	au_id	au_fname	au_lname	state
1	1	Bill	Gates	UT
2	2	Mark	Zuckerberg	UT
3	3	Elon	Musk	NULL



触发器 – Instead Of 触发器



School of software Yunnan University

□ Instead of 触发器

```
Update authors
Set au_fname=au_lname Where state = 'UT'
```

%

消息

Trigger Output: 2rows were updated.

(2 行受影响)

完成时间: 2022-05-29T11:25:42.2601600+08:00

	au_id	au_fname	au_lname	state
1	1	Bill	Gates	UT
2	2	Mark	Zuckerberg	UT
3	3	Elon	Musk	NULL



触发器



School of software Yunnan University

□ 什么时候使用**Instead of**触发器

- 数据库里的数据禁止修改
- 有可能要回滚修改的**SQL** 语句
- 在视图中使用的触发器，**After**触发器不能在视图中使用
- 控制数据的修改方式和流程

